HOSPITAL MANAGEMENT SYSTEM (HMS)

DELIVERABLE PART-3: TEST CASE DOCUMENT

Deliverable Part-3: Updated Test Case Document (TCD v2.0)

Comprehensive Test Specifications Reflecting Sprint 1 & Sprint 2 Implementations

---

Document Identifier: TCD-HMS-2026-V2.0

Document Version: 2.0 (Post-Sprint 1 & Sprint 2 Baseline Update)

Date: November 2026

Course: Software Engineering Mini-Project

Institution: Department of Computer Science and Engineering, PES University

Project Team:

- **Hrushikesh Karnam** (SRN: PES1UG24CS668) — QA Lead

- **Muruli H** (SRN: PES1UG24CS676) — Test Engineer

- **Mandagandla Jhansi** (SRN: PES1UG24CS672) — Developer Lead

- **Neeta R** (SRN: PES1UG24CS680) — Support Lead

---

1. Document Control and Revision History

Version	Date	Author(s)	Description of Changes	Approved By
1.0	20-Oct-2026	QA Lead & Test Engineer	Initial Software Test Plan (STP-HMS-2026-V1.0) baseline containing 54 requirements and 10 representative test case specifications. *(Note: Cover page printed as '20 November 2026'; internal revision records cite 20-Oct-2026).*	Project Team
2.0	02-Nov-2026	Full Project Team	Comprehensive Deliverable Part-3 update. Examined all implementations merged in Sprint 1 (PRs #60–#90) and Sprint 2 (PRs #91–#127). Expanded test specifications from 10 representative cases to 60 exhaustive detailed test cases (54 baseline + 6 supplementary sprint additions). Added immediate field retry rules, duplicate identity heuristics, cancellation confirmation workflows, and sanitized error assertions.	Pending Course Evaluator Review

Baseline Date Note: On the title page of the baseline Software Test Plan PDF (`DOCS/Software_Test_Plan.pdf`), the document date is printed as "Date: 20 November 2026". However, its internal document control table, revision history, and all subsequent Sprint 1 and Sprint 2 reports consistently establish its creation date as 20 October 2026. This updated document maintains traceability to that October 2026 baseline.

---

2. Introduction

The **Hospital Management System (HMS)** is a standalone, single-user C++ console application designed for hospital administrative staff to manage patient records, doctor profiles, and appointment schedules using local CSV flat-file persistence.

This document represents the formal **Updated Test Case Document (TCD v2.0)** for Deliverable Part-3. It operationalizes the requirements set forth in the *Software Requirements Specification (SRS v1.0)* and *Software Test Plan (STP v1.0)*, incorporating all architectural, functional, non-functional, and security hardening delivered across Sprint 1 and Sprint 2.

---

3. Purpose and Scope

3.1 Purpose

The purpose of this document is to define exact, reproducible, and verifiable test case specifications for validating the complete functional and quality scope of HMS. Each test case provides precise preconditions, input data, execution steps, and exact expected application outputs.

3.2 Scope

- **In Scope:**

- Patient Management (`HMS-F-001` to `HMS-F-007`)

- Doctor Management (`HMS-F-008` to `HMS-F-014`)

- Appointment Management & Conflict Detection (`HMS-F-015` to `HMS-F-024`)

- Persistence & Resilient File Handling (`HMS-F-025` to `HMS-F-031`)

- Console User Interface Navigation (`HMS-F-032` to `HMS-F-036`)

- Non-Functional Quality Attributes (`HMS-NF-001` to `HMS-NF-010`)

- Security Safeguards & Privacy Minimization (`HMS-SR-001` to `HMS-SR-008`)

- Supplementary Sprint Capabilities (`HMS-SR-009`, `HMS-SR-010`, `HMS-F-022-EXT`, `HMS-F-001/008-EXT`, `HMS-F-023-EXT`, `HMS-F-003/010/018-EXT`)

- **Out of Scope:** Billing, pharmacy inventory, laboratory clinical charts, external database servers (SQL), multi-user concurrency, and network communications.

---

4. References

1. *HMS Software Requirements Specification (SRS)* v1.0, September 2026.

2. *HMS Software Test Plan (STP)* v1.0, Document ID: STP-HMS-2026-V1.0, October 2026.

3. *HMS System Architecture Document (SAD)* v1.0, October 2026.

4. *IEEE Std 829-2008*: IEEE Standard for Software and System Test Documentation.

5. *IEEE Std 830-1998*: IEEE Recommended Practice for Software Requirements Specifications.

6. GitHub Repository Merged Pull Requests #60–#127, October 2026.

---

5. Test Strategy and Test Levels

Testing is conducted across four structured levels:

1. **Unit Testing:** Focuses on domain entity validation functions (`Patient::validate*`, `Doctor::validate*`, `Appointment::validate*`), text sanitization rules (VR-01 through VR-14), and isolated parsing algorithms.

2. **Integration Testing:** Focuses on interactions between `HospitalSystem` business workflows and `FileManager` serialization/deserialization routines.

3. **System Testing:** End-to-end black-box verification of compiled `hms.exe` via console menu simulation, verifying state transitions, process exit codes, and disk file updates.

4. **Acceptance Testing (UAT):** Execution of clinical workflow scenarios ensuring zero memory corruption, robust error feedback, and compliance with hospital administrative expectations.

---

6. Test Environment

Component	Specification	Purpose
Operating System	Windows 11 (64-bit) / Ubuntu Linux 22.04 LTS	Target execution platforms (`HMS-NF-007`)
Compiler Toolchain	MinGW-w64 GCC 9+ / MSVC 19.44 (`-std=c++17 -Wall`)	Clean compilation and binary generation
Build System	CMake 3.20+ & CTest	Automated test target orchestration
Persistent Storage	Local CSV flat files (`patients.csv`, `doctors.csv`, `appointments.csv`)	Flat-file persistent database
Test Execution Shell	Windows PowerShell 5.1+ / Command Prompt / Bash	Test harness driver execution

---

7. Test Data

Test data sets are partitioned into:

- **Valid Nominal Data:** Standard patient demographics, specialist doctors, and non-conflicting booking slots.

- **Boundary Test Data:** Min/max age boundaries (0, 120), contact lengths (exactly 10 digits), maximum field lengths (e.g., 50-char names, 100-char addresses).

- **Invalid & Malformed Data:** Calendar boundary errors (leap years, 30-day months), invalid time formats (`24:00`, `9:5`), formula injection prefixes (`=`,`+`,`-`,`@`), and embedded commas/newlines.

- **Stress & Volumetric Datasets:** 10,000-character over-length input strings and 1,000 synthetic record CSV files for performance benchmarks.

---

8. Detailed Patient Management Test Cases (HMS-F-001 to HMS-F-007)

Test Case Specification: TC-PAT-01 — Register Patient with Complete Profile

Attribute	Specification Detail
Test Case ID	`TC-PAT-01`
Requirement ID	`HMS-F-001`
Title	Register Patient with Complete Profile
Test Objective	Verify successful patient registration with valid demographic data and immediate CSV persistence.
Test Type	Positive Functional
Test Level	System / Integration
Preconditions	hms.exe is running; Patient P001 is not registered in patients.csv.
Test Data	`ID: P001, Name: John Doe, Age: 45, Gender: M, Contact: 9876543210, Address: 123 Main St, Note: Hypertension`
Execution Steps	1. Select Patient Management -> 1 Register Patient. 2. Enter Patient ID 'P001'. 3. Enter Full Name 'John Doe'. 4. Enter Age '45'. 5. Enter Gender 'M'. 6. Enter Contact '9876543210'. 7. Enter Address '123 Main St'. 8. Enter Medical Note 'Hypertension'.
Expected Result	System displays registration confirmation message; patients.csv immediately contains the new record with all 7 fields.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #61, #80. In CTest exercised via TC-PAT-05_PatientDetailsAndRegistrationPersistence.

Test Case Specification: TC-PAT-02 — Reject Duplicate Patient ID (Case-Insensitive)

Attribute	Specification Detail
Test Case ID	`TC-PAT-02`
Requirement ID	`HMS-F-002`
Title	Reject Duplicate Patient ID (Case-Insensitive)
Test Objective	Verify that attempting to register an existing Patient ID is rejected case-insensitively and leaves records unchanged.
Test Type	Negative Functional
Test Level	Unit / Integration
Preconditions	Patient P001 is already registered in patients.csv.
Test Data	`ID: p001, Name: Jane Smith, Age: 30, Gender: F, Contact: 9123456789, Address: 456 Park Ave, Note: None`
Execution Steps	1. Select Patient Management -> 1 Register Patient. 2. Enter Patient ID 'p001' (lowercase). 3. Observe system validation check.
Expected Result	System displays exact error 'Patient ID already exists'; operation halts or prompts for re-entry; patients.csv remains unmodified.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #62. CTest target: TC-PAT-02_DuplicatePatientId.

Test Case Specification: TC-PAT-03 — Validate Patient Fields and Immediate Retry

Attribute	Specification Detail	
Test Case ID	`TC-PAT-03`	
Requirement ID	`HMS-F-003`	
Title	Validate Patient Fields and Immediate Retry	
Test Objective	Verify comprehensive field validation against rules VR-01 to VR-07, VR-14, and prompt retry on error.	
Test Type	Boundary / Security	
Test Level	Unit	
Preconditions	Application running at Patient Registration prompt.	
Test Data	`Age: 121, Gender: X, Contact: 12345, Name: '=cmd	' /C calc'!A0`
Execution Steps	1. Attempt entering Age '121' (exceeds 120). Observe error. 2. Re-enter Age '45'. 3. Attempt entering Gender 'X'. Observe error. 4. Re-enter Gender 'M'. 5. Attempt entering Contact '12345' (not 10 digits). Observe error. 6. Re-enter Contact '9876543210'.	
Expected Result	Invalid entries are rejected immediately with specific error message naming violated rule; system re-prompts for same field without aborting valid fields.	
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)	
Execution Status	**Not Executed**	
Remarks / Evidence	PR #63, #127. CTest target: TC-PAT-03_ValidatePatientFields.	

Test Case Specification: TC-PAT-04 — Search Patients by ID or Name with Whitespace Trimming

Attribute	Specification Detail
Test Case ID	`TC-PAT-04`
Requirement ID	`HMS-F-004`
Title	Search Patients by ID or Name with Whitespace Trimming
Test Objective	Verify patient search by exact Patient ID and partial name match, ensuring whitespace padding is trimmed.
Test Type	Positive / Negative Functional
Test Level	System
Preconditions	Patient P001 (John Doe) and P002 (Jane Smith) are registered in patients.csv.
Test Data	`Search 1: ' P001 ', Search 2: 'doe', Search 3: 'Unknown'`
Execution Steps	1. Select Patient Management -> 2 Search Patient. 2. Select Search by ID and enter ' P001 '. 3. Select Search by Name and enter 'doe'. 4. Select Search by Name and enter 'Unknown'.
Expected Result	Search 1 returns Patient P001. Search 2 returns Patient P001. Search 3 displays 'No matching patient found'.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #67, #70. CTest target: TC-PAT-04_SearchPatients.

Test Case Specification: TC-PAT-05 — View Patient Details and Full Patient Listing

Attribute	Specification Detail
Test Case ID	`TC-PAT-05`
Requirement ID	`HMS-F-005`
Title	View Patient Details and Full Patient Listing
Test Objective	Verify display of all 7 stored fields for a selected patient and formatted listing of all registered patients.
Test Type	Positive Functional
Test Level	System
Preconditions	Patients P001 and P002 are registered in system.
Test Data	`Option 3 View All Patients, followed by selecting Patient P001.`
Execution Steps	1. Select Patient Management -> 3 View All. 2. Inspect listed columns. 3. Enter Patient ID 'P001' to view full details.
Expected Result	List view displays Patient ID, Name, Age, Gender, Contact for all records. Detailed view displays all 7 fields exactly as stored.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #68, #71. CTest target: TC-PAT-05_PatientDetailsAndRegistrationPersistence.

Test Case Specification: TC-PAT-06 — Update Patient Details with Interactive Validation

Attribute	Specification Detail
Test Case ID	`TC-PAT-06`
Requirement ID	`HMS-F-006`
Title	Update Patient Details with Interactive Validation
Test Objective	Verify updating existing patient fields (contact, address, note) with immediate validation and disk persistence.
Test Type	Positive / Negative Functional
Test Level	System / Integration
Preconditions	Patient P001 exists with contact '9876543210' and address '123 Main St'.
Test Data	`New Contact: '9998887776', New Address: '789 Broadway', Invalid Contact: '123'`
Execution Steps	1. Select Patient Management -> 4 Update Patient. 2. Enter Patient ID 'P001'. 3. Choose field to update (e.g. Contact). 4. Enter invalid contact '123'. Observe rejection. 5. Enter valid contact '9998887776'. 6. Save update.
Expected Result	Invalid values rejected with error; valid changes updated in memory and committed to patients.csv immediately.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #85, #127. CTest target: TC-PAT-06_UpdatePatientDetails.

Test Case Specification: TC-PAT-07 — Prevent Patient ID Editing during Update

Attribute	Specification Detail
Test Case ID	`TC-PAT-07`
Requirement ID	`HMS-F-007`
Title	Prevent Patient ID Editing during Update
Test Objective	Verify that Patient ID remains immutable during record update and updating nonexistent ID is rejected.
Test Type	Security / Integrity
Test Level	System
Preconditions	Patient P001 is registered; P999 is not registered.
Test Data	`ID: P999`
Execution Steps	1. Select Patient Management -> 4 Update Patient. 2. Enter nonexistent Patient ID 'P999'. 3. Observe system response. 4. Enter valid ID 'P001' and inspect offered update options.
Expected Result	Entering P999 displays exact message 'Patient not found'. Update menu offers options for name, age, gender, contact, address, note; no option exists to modify Patient ID.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #86. CTest target: TC-PAT-07_PreventPatientIdEdits.

9. Detailed Doctor Management Test Cases (HMS-F-008 to HMS-F-014)

Test Case Specification: TC-DOC-01 — Add Doctor with Complete Profile

Attribute	Specification Detail
Test Case ID	`TC-DOC-01`
Requirement ID	`HMS-F-008`
Title	Add Doctor with Complete Profile
Test Objective	Verify successful addition of doctor profile with all 6 fields and immediate persistence to doctors.csv.
Test Type	Positive Functional
Test Level	System / Integration
Preconditions	Doctor D001 is not registered in doctors.csv.
Test Data	`ID: D001, Name: Dr. Alice Smith, Spec: Cardiology, Dept: Cardio, Contact: 9876543211, Avail: Mon-Fri 09:00-17:00`
Execution Steps	1. Select Doctor Management -> 1 Add Doctor. 2. Enter Doctor ID 'D001'. 3. Enter Name 'Dr. Alice Smith'. 4. Enter Specialization 'Cardiology'. 5. Enter Department 'Cardio'. 6. Enter Contact '9876543211'. 7. Enter Availability 'Mon-Fri 09:00-17:00'.
Expected Result	Confirmation displayed; doctors.csv contains the new doctor record with all 6 fields.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #64. CTest target: TC-DOC-01_AddDoctor.

Test Case Specification: TC-DOC-02 — Reject Duplicate Doctor ID (Case-Insensitive)

Attribute	Specification Detail
Test Case ID	`TC-DOC-02`
Requirement ID	`HMS-F-009`
Title	Reject Duplicate Doctor ID (Case-Insensitive)
Test Objective	Verify that attempting to add an existing Doctor ID is rejected case-insensitively and leaves records unchanged.
Test Type	Negative Functional
Test Level	Unit / Integration
Preconditions	Doctor D001 is already registered in doctors.csv.
Test Data	`ID: d001, Name: Dr. Bob Jones, Spec: Pediatrics, Dept: Peds, Contact: 9123456788, Avail: Tue-Sat 10:00-16:00`
Execution Steps	1. Select Doctor Management -> 1 Add Doctor. 2. Enter Doctor ID 'd001' (lowercase). 3. Observe system response.
Expected Result	System displays exact error 'Doctor ID already exists'; registration halts; doctors.csv remains unmodified.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #65. CTest target: TC-DOC-02_RejectDuplicateDoctorId.

Test Case Specification: TC-DOC-03 — Validate Doctor Fields and Formatting

Attribute	Specification Detail
Test Case ID	`TC-DOC-03`
Requirement ID	`HMS-F-010`
Title	Validate Doctor Fields and Formatting
Test Objective	Verify validation of doctor fields against rules VR-01, VR-02, VR-05, VR-08, VR-09, and VR-14.
Test Type	Boundary / Validation
Test Level	Unit
Preconditions	Application running at Add Doctor prompt.
Test Data	`Spec: >40 chars, Contact: 12345, Availability: >60 chars, Name containing comma`
Execution Steps	1. Enter Doctor ID 'D002'. 2. Attempt invalid Specialization (>40 chars). Observe error. 3. Re-enter 'Neurology'. 4. Attempt invalid contact '12345'. Observe error. 5. Re-enter '9876543212'.
Expected Result	Invalid entries rejected immediately naming violated rule; system re-prompts for same field without aborting registration.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #66, #127. CTest target: TC-DOC-03_ValidateDoctorFields.

Test Case Specification: TC-DOC-04 — Search Doctors by ID, Name, or Specialization

Attribute	Specification Detail
Test Case ID	`TC-DOC-04`
Requirement ID	`HMS-F-011`
Title	Search Doctors by ID, Name, or Specialization
Test Objective	Verify doctor search across exact ID, partial name, and partial specialization with case-insensitivity.
Test Type	Positive / Negative Functional
Test Level	System
Preconditions	Doctor D001 (Dr. Alice Smith, Cardiology) registered in system.
Test Data	`Search 1: 'D001', Search 2: 'alice', Search 3: 'cardio', Search 4: 'Oncology'`
Execution Steps	1. Select Doctor Management -> 2 Search Doctor. 2. Search by ID 'D001'. 3. Search by Name 'alice'. 4. Search by Specialization 'cardio'. 5. Search by Specialization 'Oncology'.
Expected Result	Searches 1, 2, and 3 return Dr. Alice Smith. Search 4 displays 'No matching doctor found'.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #72. CTest target: TC-DOC-04_SearchDoctors.

Test Case Specification: TC-DOC-05 — View Doctor Details and Full Doctor Listing

Attribute	Specification Detail
Test Case ID	`TC-DOC-05`
Requirement ID	`HMS-F-012`
Title	View Doctor Details and Full Doctor Listing
Test Objective	Verify display of all 6 stored fields for a selected doctor and full doctor listing.
Test Type	Positive Functional
Test Level	System
Preconditions	Doctors D001 and D002 are registered in system.
Test Data	`Option 3 View All Doctors, followed by selecting D001.`
Execution Steps	1. Select Doctor Management -> 3 View All. 2. Inspect doctor table. 3. Enter Doctor ID 'D001' to view full record.
Expected Result	Listing shows Doctor ID, Name, Specialization, Department. Full detail view displays all 6 stored fields.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #73. CTest target: TC-DOC-05_ViewAndListDoctors.

Test Case Specification: TC-DOC-06 — Update Doctor Details with Field Validation

Attribute	Specification Detail
Test Case ID	`TC-DOC-06`
Requirement ID	`HMS-F-013`
Title	Update Doctor Details with Field Validation
Test Objective	Verify updating existing doctor details (department, contact, availability) with immediate validation.
Test Type	Positive / Negative Functional
Test Level	System / Integration
Preconditions	Doctor D001 is registered in system.
Test Data	`New Department: 'Advanced Cardio', New Availability: 'Mon-Thu 08:00-14:00'`
Execution Steps	1. Select Doctor Management -> 4 Update Doctor. 2. Enter Doctor ID 'D001'. 3. Select Department and enter 'Advanced Cardio'. 4. Select Availability and update. 5. Confirm update.
Expected Result	Valid changes reflected in memory and saved to doctors.csv immediately; invalid inputs rejected with immediate retry.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #87, #127. CTest target: TC-DOC-06_UpdateDoctorDetails.

Test Case Specification: TC-DOC-07 — Prevent Doctor ID Editing during Update

Attribute	Specification Detail
Test Case ID	`TC-DOC-07`
Requirement ID	`HMS-F-014`
Title	Prevent Doctor ID Editing during Update
Test Objective	Verify that Doctor ID cannot be modified during record update and nonexistent ID update is rejected.
Test Type	Security / Integrity
Test Level	System
Preconditions	Doctor D001 is registered; D999 is not registered.
Test Data	`ID: D999`
Execution Steps	1. Select Doctor Management -> 4 Update Doctor. 2. Enter nonexistent Doctor ID 'D999'. 3. Observe system response. 4. Enter valid ID 'D001' and inspect update options.
Expected Result	Entering D999 displays exact message 'Doctor not found'. Update menu provides no option to edit Doctor ID.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #88. CTest target: TC-DOC-07_PreventDoctorIdEdits.

10. Detailed Appointment Management Test Cases (HMS-F-015 to HMS-F-024)

Test Case Specification: TC-APT-01 — Book Appointment with Generated ID and Initial Status

Attribute	Specification Detail
Test Case ID	`TC-APT-01`
Requirement ID	`HMS-F-015`
Title	Book Appointment with Generated ID and Initial Status
Test Objective	Verify appointment booking creates unique ID (A0001+) with status Booked and persists to appointments.csv.
Test Type	Positive Functional
Test Level	System / Integration
Preconditions	Patient P001 and Doctor D001 exist; no conflicting bookings exist.
Test Data	`Patient: P001, Doctor: D001, Date: 2026-11-15, Time: 10:00`
Execution Steps	1. Select Appointment Management -> 1 Book Appointment. 2. Enter Patient ID 'P001'. 3. Enter Doctor ID 'D001'. 4. Enter Date '2026-11-15'. 5. Enter Time '10:00'.
Expected Result	Booking succeeds displaying generated Appointment ID (e.g. A0001); appointments.csv contains row with status Booked.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #69. CTest target: TC-APT-01_BookAppointment.

Test Case Specification: TC-APT-02 — Reject Appointment Booking for Unknown Patient

Attribute	Specification Detail
Test Case ID	`TC-APT-02`
Requirement ID	`HMS-F-016`
Title	Reject Appointment Booking for Unknown Patient
Test Objective	Verify booking rejection when Patient ID does not exist in registered patient records.
Test Type	Negative Functional
Test Level	Integration
Preconditions	Doctor D001 exists; Patient P999 does not exist.
Test Data	`Patient: P999, Doctor: D001, Date: 2026-11-15, Time: 10:00`
Execution Steps	1. Select Appointment Management -> 1 Book Appointment. 2. Enter Patient ID 'P999'. 3. Observe system response.
Expected Result	System displays exact message 'Patient not found'; booking aborted or prompts retry; no appointment record created.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #74. CTest target: TC-APT-02_RejectUnknownPatient.

Test Case Specification: TC-APT-03 — Reject Appointment Booking for Unknown Doctor

Attribute	Specification Detail
Test Case ID	`TC-APT-03`
Requirement ID	`HMS-F-017`
Title	Reject Appointment Booking for Unknown Doctor
Test Objective	Verify booking rejection when Doctor ID does not exist in registered doctor records.
Test Type	Negative Functional
Test Level	Integration
Preconditions	Patient P001 exists; Doctor D999 does not exist.
Test Data	`Patient: P001, Doctor: D999, Date: 2026-11-15, Time: 10:00`
Execution Steps	1. Select Appointment Management -> 1 Book Appointment. 2. Enter Patient ID 'P001'. 3. Enter Doctor ID 'D999'. 4. Observe system response.
Expected Result	System displays exact message 'Doctor not found'; booking aborted or prompts retry; no appointment record created.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #75. CTest target: TC-APT-03_RejectUnknownDoctor.

Test Case Specification: TC-APT-04 — Validate Appointment Date and Time Formats

Attribute	Specification Detail
Test Case ID	`TC-APT-04`
Requirement ID	`HMS-F-018`
Title	Validate Appointment Date and Time Formats
Test Objective	Verify rejection of invalid calendar dates, leap year violations, and out-of-range times.
Test Type	Boundary / Validation
Test Level	Unit
Preconditions	Patient P001 and Doctor D001 exist.
Test Data	`Date 1: 2026-02-29 (invalid non-leap), Date 2: 2024-02-29 (valid leap), Time 1: 24:00, Time 2: 09:30`
Execution Steps	1. Attempt booking with Date '2026-02-29'. Observe rejection. 2. Enter valid Date '2026-11-15'. 3. Attempt booking with Time '24:00'. Observe rejection. 4. Enter valid Time '09:30'.
Expected Result	Invalid dates and times rejected with format error message; valid calendar inputs accepted.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #76, #127. CTest target: TC-APT-04_ValidateAppointmentDateTime.

Test Case Specification: TC-APT-05 — Reject Doctor Double-Booking Conflict

Attribute	Specification Detail
Test Case ID	`TC-APT-05`
Requirement ID	`HMS-F-019`
Title	Reject Doctor Double-Booking Conflict
Test Objective	Verify rejection of second appointment booking for same doctor at same date and time.
Test Type	Logic / Conflict
Test Level	Integration
Preconditions	Doctor D001 has active appointment A0001 on 2026-11-15 at 10:00 with Patient P001.
Test Data	`Patient: P002, Doctor: D001, Date: 2026-11-15, Time: 10:00`
Execution Steps	1. Select Appointment Management -> 1 Book Appointment. 2. Enter Patient ID 'P002'. 3. Enter Doctor ID 'D001'. 4. Enter Date '2026-11-15', Time '10:00'.
Expected Result	System rejects booking with exact message 'Doctor not available at the requested date and time'; appointment count unchanged.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #91. CTest target: TC-APT-05_TC-NFR-08_PreventDoubleBooking.

Test Case Specification: TC-APT-06 — Check Doctor Availability Query

Attribute	Specification Detail
Test Case ID	`TC-APT-06`
Requirement ID	`HMS-F-020`
Title	Check Doctor Availability Query
Test Objective	Verify informational query reporting doctor availability without modifying records.
Test Type	Positive Functional
Test Level	System
Preconditions	Doctor D001 booked on 2026-11-15 at 10:00; slot 11:00 is free.
Test Data	`Slot 1: D001, 2026-11-15 10:00; Slot 2: D001, 2026-11-15 11:00`
Execution Steps	1. Select Appointment Management -> 4 Check Doctor Availability. 2. Query D001 at 2026-11-15 10:00. 3. Query D001 at 2026-11-15 11:00.
Expected Result	Query 1 reports 'Not available'. Query 2 reports 'Available'. CSV files remain unmodified.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #92. CTest target: TC-APT-06_CheckDoctorAvailability.

Test Case Specification: TC-APT-07 — Exclude Cancelled Appointments from Conflicts

Attribute	Specification Detail
Test Case ID	`TC-APT-07`
Requirement ID	`HMS-F-021`
Title	Exclude Cancelled Appointments from Conflicts
Test Objective	Verify that cancelling an appointment frees the slot, allowing re-booking at the same date and time.
Test Type	Logic / Conflict
Test Level	Integration
Preconditions	Doctor D001 has active appointment A0001 on 2026-11-15 at 10:00.
Test Data	`Step 1: Cancel A0001. Step 2: Book P002 with D001 on 2026-11-15 at 10:00.`
Execution Steps	1. Cancel appointment A0001. 2. Attempt booking P002 with D001 at 2026-11-15 10:00.
Expected Result	Booking succeeds with new Appointment ID A0002; cancelled slot is successfully reused.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #93. CTest target: TC-APT-07_IgnoreCancelledAppointmentsInConflicts.

Test Case Specification: TC-APT-08 — View Appointments in Chronological Order

Attribute	Specification Detail
Test Case ID	`TC-APT-08`
Requirement ID	`HMS-F-022`
Title	View Appointments in Chronological Order
Test Objective	Verify appointment views list records in ascending date and time order across all view filters.
Test Type	Positive Functional
Test Level	System
Preconditions	Three appointments seeded: A001 (2026-11-20 09:00), A002 (2026-11-10 14:00), A003 (2026-11-10 09:30).
Test Data	`View All Appointments`
Execution Steps	1. Select Appointment Management -> 2 View Appointments -> 1 View All. 2. Inspect order of displayed records.
Expected Result	Records appear sorted chronologically: A003 (2026-11-10 09:30) precedes A002 (2026-11-10 14:00), which precedes A001 (2026-11-20 09:00).
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #94. CTest target: TC-APT-08_ViewAppointmentsDateTimeOrder.

Test Case Specification: TC-APT-09 — Cancel Appointment with Confirmation and Retain Record

Attribute	Specification Detail
Test Case ID	`TC-APT-09`
Requirement ID	`HMS-F-023`
Title	Cancel Appointment with Confirmation and Retain Record
Test Objective	Verify cancelling appointment updates status to Cancelled and preserves row in appointments.csv.
Test Type	Positive Functional
Test Level	System / Integration
Preconditions	Active appointment A0001 exists with status Booked.
Test Data	`Appointment ID: A0001, Confirm: Y`
Execution Steps	1. Select Appointment Management -> 3 Cancel Appointment. 2. Enter Appointment ID 'A0001'. 3. Confirm cancellation with 'Y'.
Expected Result	Status updated to Cancelled; record retained in appointments.csv with total row count unchanged.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #95, #122. CTest target: TC-APT-09_CancelAppointment.

Test Case Specification: TC-APT-10 — Reject Invalid Appointment Cancellation

Attribute	Specification Detail
Test Case ID	`TC-APT-10`
Requirement ID	`HMS-F-024`
Title	Reject Invalid Appointment Cancellation
Test Objective	Verify rejection when attempting to cancel nonexistent or already-cancelled appointment ID.
Test Type	Negative Functional
Test Level	System
Preconditions	Appointment A0001 is Cancelled; A9999 does not exist.
Test Data	`Case 1: A9999, Case 2: A0001`
Execution Steps	1. Select Appointment Management -> 3 Cancel Appointment. 2. Enter nonexistent ID 'A9999'. Observe rejection. 3. Enter already-cancelled ID 'A0001'. Observe rejection.
Expected Result	Case 1 displays 'Appointment not found'. Case 2 displays 'Appointment already cancelled'. Records remain unchanged.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #96, #127. CTest target: TC-APT-10_RejectInvalidCancellation.

11. Detailed Persistence & File Handling Test Cases (HMS-F-025 to HMS-F-031)

Test Case Specification: TC-PER-01 — Load Pre-Existing CSV Records at Startup

Attribute	Specification Detail
Test Case ID	`TC-PER-01`
Requirement ID	`HMS-F-025`
Title	Load Pre-Existing CSV Records at Startup
Test Objective	Verify all records from patients.csv, doctors.csv, and appointments.csv load into memory on startup.
Test Type	Functional / Integration
Test Level	System / Integration
Preconditions	CSV files seeded with 5 patients, 3 doctors, and 4 appointments.
Test Data	`Standard launch of hms.exe`
Execution Steps	1. Launch hms.exe. 2. Navigate to View All Patients, Doctors, and Appointments.
Expected Result	All 5 patients, 3 doctors, and 4 appointments are visible immediately with correct field values.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #77. CTest target: TC-PER-01_StartupCSVLoading.

Test Case Specification: TC-PER-02 — Graceful Startup with Missing CSV Files

Attribute	Specification Detail
Test Case ID	`TC-PER-02`
Requirement ID	`HMS-F-026`
Title	Graceful Startup with Missing CSV Files
Test Objective	Verify application starts normally when CSV files are missing, treating them as empty and creating on save.
Test Type	Robustness
Test Level	System
Preconditions	patients.csv, doctors.csv, and appointments.csv are deleted from working directory.
Test Data	`Standard launch, followed by registering Patient P001.`
Execution Steps	1. Delete all CSV files. 2. Launch hms.exe. Verify main menu displays. 3. Register Patient P001 and exit.
Expected Result	Application launches without errors treating datasets as empty; patients.csv is created on first save.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #78. CTest target: TC-PER-02_MissingCSVFiles.

Test Case Specification: TC-PER-03 — Skip Malformed CSV Rows during Startup Loading

Attribute	Specification Detail
Test Case ID	`TC-PER-03`
Requirement ID	`HMS-F-027`
Title	Skip Malformed CSV Rows during Startup Loading
Test Objective	Verify corrupted CSV lines are skipped safely while valid lines are loaded, reporting skipped count.
Test Type	Robustness / Security
Test Level	Integration
Preconditions	CSV file contains 10 valid records and 3 malformed rows (wrong column count, invalid age, bad format).
Test Data	`Launch application with corrupted dataset.`
Execution Steps	1. Seed CSV with 10 valid and 3 malformed rows. 2. Launch application and observe startup messages. 3. Inspect loaded records.
Expected Result	10 valid records are loaded; system displays '3 malformed record(s) skipped'; application remains stable.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #79, #112. CTest target: TC-PER-03_SkipMalformedCSV.

Test Case Specification: TC-PER-04 — Skip Duplicate and Orphan Records during Load

Attribute	Specification Detail
Test Case ID	`TC-PER-04`
Requirement ID	`HMS-F-028`
Title	Skip Duplicate and Orphan Records during Load
Test Objective	Verify duplicate entity IDs and orphan appointments are skipped during startup CSV parsing.
Test Type	Functional / Integrity
Test Level	Integration
Preconditions	CSV files contain duplicate Patient P001 rows and appointment referencing nonexistent Doctor D999.
Test Data	`Launch application with duplicate/orphan CSV dataset.`
Execution Steps	1. Seed duplicate Patient ID and orphan appointment. 2. Launch application. 3. Query patient list and appointment list.
Expected Result	Duplicate patient row and orphan appointment row are excluded from active memory and counted as skipped.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #97. In CTest registered as TC-PER-08_SkipDuplicateAndOrphanRecords.

Test Case Specification: TC-PER-05 — Immediate CSV Persistence after Modifying Operations

Attribute	Specification Detail
Test Case ID	`TC-PER-05`
Requirement ID	`HMS-F-029`
Title	Immediate CSV Persistence after Modifying Operations
Test Objective	Verify that data files are updated immediately after every add, update, booking, or cancellation.
Test Type	Functional / Integration
Test Level	System / Integration
Preconditions	Application running at main menu.
Test Data	`Register Patient P001, Add Doctor D001, Book Appointment A0001.`
Execution Steps	1. Register Patient P001. 2. Without exiting, inspect patients.csv on disk. 3. Book appointment and inspect appointments.csv.
Expected Result	Disk files contain new records immediately without requiring application exit.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #80. In CTest exercised via TC-PAT-05_PatientDetailsAndRegistrationPersistence.

Test Case Specification: TC-PER-06 — 100% Data Retention across Normal Exit and Restart

Attribute	Specification Detail
Test Case ID	`TC-PER-06`
Requirement ID	`HMS-F-030`
Title	100% Data Retention across Normal Exit and Restart
Test Objective	Verify complete data durability across process termination and subsequent restart.
Test Type	Persistence / Durability
Test Level	System
Preconditions	Application populated with patients, doctors, and appointments.
Test Data	`Exit application via Option 4; relaunch process.`
Execution Steps	1. Verify record counts and field values before exit. 2. Exit application cleanly (Option 4). 3. Relaunch hms.exe and inspect records.
Expected Result	100% of confirmed records and appointment statuses are preserved identically.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #81, #101. In CTest verified via TC-NFR-03_DataRetention.

Test Case Specification: TC-PER-07 — Handle CSV Write Failure with Non-Destructive Atomic Safety

Attribute	Specification Detail
Test Case ID	`TC-PER-07`
Requirement ID	`HMS-F-031`
Title	Handle CSV Write Failure with Non-Destructive Atomic Safety
Test Objective	Verify graceful error reporting and preservation of existing file content when CSV write fails.
Test Type	Reliability / Security
Test Level	Integration
Preconditions	patients.csv set to read-only permissions (chmod 444 / Read-only attribute).
Test Data	`Attempt registering Patient P002.`
Execution Steps	1. Mark patients.csv read-only. 2. Attempt registering Patient P002. 3. Inspect console error and file content.
Expected Result	System displays write failure error; previous file content remains intact; in-memory menu remains usable.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #89, #113. CTest target: TC-PER-07_HandleCSVWriteFailures.

12. Detailed Console UI Test Cases (HMS-F-032 to HMS-F-036)

Test Case Specification: TC-UI-01 — Console Menu Navigation Hierarchy

Attribute	Specification Detail
Test Case ID	`TC-UI-01`
Requirement ID	`HMS-F-032`
Title	Console Menu Navigation Hierarchy
Test Objective	Verify main menu and 3 submenus navigate correctly and Option 0 returns to previous menu level.
Test Type	Positive Functional
Test Level	System
Preconditions	Application running at Main Menu.
Test Data	`Navigation sequence: 1 -> 0 -> 2 -> 0 -> 3 -> 0`
Execution Steps	1. Press 1 (Patient Management). Verify submenu. 2. Press 0 (Return). Verify Main Menu. 3. Press 2 (Doctor Management). Verify submenu. 4. Press 0. Press 3 (Appointment Management). Press 0.
Expected Result	All submenus display correct operations; Option 0 returns reliably to parent menu.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #82. CTest target: TC-UI-01_MenuNavigation.

Test Case Specification: TC-UI-02 — Reject Invalid Menu Options

Attribute	Specification Detail
Test Case ID	`TC-UI-02`
Requirement ID	`HMS-F-033`
Title	Reject Invalid Menu Options
Test Objective	Verify rejection of empty, alphabetic, negative, and out-of-range menu selections.
Test Type	Robustness
Test Level	System
Preconditions	Application running at Main Menu prompt.
Test Data	`Inputs: '', 'abc', '-1', '9', '999999'`
Execution Steps	1. Enter empty string. Observe response. 2. Enter 'abc'. Observe response. 3. Enter '-1'. Observe response. 4. Enter '9'. Observe response.
Expected Result	System displays invalid choice error message and re-displays menu without crashing.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #83. CTest target: TC-UI-02_RejectInvalidMenuOptions.

Test Case Specification: TC-UI-03 — Transactional State Protection after Failed Operations

Attribute	Specification Detail
Test Case ID	`TC-UI-03`
Requirement ID	`HMS-F-034`
Title	Transactional State Protection after Failed Operations
Test Objective	Verify that in-memory and disk records remain unaltered when an operation input fails validation.
Test Type	Integrity
Test Level	System / Integration
Preconditions	Application state snapshot taken prior to test.
Test Data	`Attempt registering patient with invalid contact number.`
Execution Steps	1. Capture initial record counts and file byte content. 2. Initiate patient registration and enter invalid contact. 3. Abort operation. 4. Re-check record counts and file byte content.
Expected Result	No partial patient record is stored in memory or written to patients.csv; state is byte-identical.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #84. No dedicated CTest target; verified across tests/test_reject_invalid_menu_options.cpp and tests/test_csv_write_failures.cpp.

Test Case Specification: TC-UI-04 — Standard Operation Outcome Messaging

Attribute	Specification Detail
Test Case ID	`TC-UI-04`
Requirement ID	`HMS-F-035`
Title	Standard Operation Outcome Messaging
Test Objective	Verify that every operation terminates with a clear success confirmation or an informative error message.
Test Type	Functional
Test Level	System
Preconditions	Application running across diverse operations.
Test Data	`Valid patient add, duplicate add, valid booking, conflict booking.`
Execution Steps	1. Execute valid operations and observe confirmation. 2. Execute failing operations and observe error messages.
Expected Result	Every operation displays exactly one clear outcome indicating success or reason for failure.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #98. CTest target: TC-UI-04_OperationOutcomeMessages.

Test Case Specification: TC-UI-05 — Clean Application Exit with Code 0

Attribute	Specification Detail
Test Case ID	`TC-UI-05`
Requirement ID	`HMS-F-036`
Title	Clean Application Exit with Code 0
Test Objective	Verify Option 4 on Main Menu terminates application cleanly with process return code 0.
Test Type	Functional
Test Level	System
Preconditions	Application running at Main Menu prompt.
Test Data	`Option 4 (Exit)`
Execution Steps	1. From Main Menu, enter '4'. 2. Observe exit message. 3. Verify OS process return code ($LASTEXITCODE).
Expected Result	Application terminates displaying exit message; process returns exit code 0.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #99. CTest targets: TC-UI-05_ExitCleanly and TC-UI-05_ExitProcessCode.

13. Detailed Non-Functional Quality Test Cases (HMS-NF-001 to HMS-NF-010)

Test Case Specification: TC-NFR-01 — Invalid Keyboard Input Crash Resilience

Attribute	Specification Detail
Test Case ID	`TC-NFR-01`
Requirement ID	`HMS-NF-001`
Title	Invalid Keyboard Input Crash Resilience
Test Objective	Verify application survives 20+ defined invalid input vectors with 0 abnormal terminations.
Test Type	Reliability
Test Level	System
Preconditions	Application running at console input prompts.
Test Data	`20 vectors: empty strings, control characters, negative integers, overflow numbers, format strings (%s%x), symbols`
Execution Steps	1. Execute automated driver feeding all 20 invalid input vectors across menus and forms. 2. Monitor process state and crash logs.
Expected Result	Zero application crashes (0% crash rate); system displays error messages and returns cleanly to prompt.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #90. CTest target: TC-NFR-01_InvalidInputCrashResilience.

Test Case Specification: TC-NFR-02 — Robust Startup under Corrupted File States

Attribute	Specification Detail
Test Case ID	`TC-NFR-02`
Requirement ID	`HMS-NF-002`
Title	Robust Startup under Corrupted File States
Test Objective	Verify application launches successfully across combinations of missing, empty, and malformed files.
Test Type	Reliability
Test Level	System
Preconditions	Combinations: all files missing; all empty; files containing 5 malformed lines each.
Test Data	`Application launch across 4 configuration permutations.`
Execution Steps	1. Test missing files startup. 2. Test empty files startup. 3. Test malformed rows startup. 4. Test mixed state startup.
Expected Result	All startup scenarios succeed; Main Menu displays without unhandled exceptions or crashes.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #100. CTest target: TC-NFR-02_RobustStartup.

Test Case Specification: TC-NFR-03 — Record Retention Rate Verification

Attribute	Specification Detail
Test Case ID	`TC-NFR-03`
Requirement ID	`HMS-NF-003`
Title	Record Retention Rate Verification
Test Objective	Verify 100% data retention rate of all confirmed records after application close and restart.
Test Type	Persistence
Test Level	System
Preconditions	10 patients, 5 doctors, and 8 appointments registered in system.
Test Data	`Exit application and relaunch.`
Execution Steps	1. Record exact field values and row counts. 2. Exit cleanly via Option 4. 3. Relaunch and verify record count and field equality.
Expected Result	Record retention rate is exactly 100%; zero data degradation or missing fields.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #101. CTest target: TC-NFR-03_DataRetention.

Test Case Specification: TC-NFR-04 — Load and Execution Latency Benchmark (1,000 Records)

Attribute	Specification Detail
Test Case ID	`TC-NFR-04`
Requirement ID	`HMS-NF-004`
Title	Load and Execution Latency Benchmark (1,000 Records)
Test Objective	Verify startup loading <= 3.0s and operations <= 2.0s with 1,000 records per file.
Test Type	Performance Benchmark
Test Level	System
Preconditions	1,000 synthetic patient records, 1,000 doctor records, and 1,000 appointment records generated.
Test Data	`1,000 synthetic rows per CSV file.`
Execution Steps	1. Measure launch time from process start to Main Menu display. 2. Measure appointment booking latency at slot 1,000. 3. Measure search latency across 1,000 records.
Expected Result	Startup file loading completes in < 3.0 seconds; booking and search operations complete in < 2.0 seconds.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #106, #125. CTest target: TC-NFR-04_Performance1000Records. (Measures in-memory / file I/O latency via displayStartupRecords(), booking, and search; full OS launch is verified manually).

Test Case Specification: TC-NFR-05 — UI Usability and Maximum Menu Depth

Attribute	Specification Detail
Test Case ID	`TC-NFR-05`
Requirement ID	`HMS-NF-005`
Title	UI Usability and Maximum Menu Depth
Test Objective	Verify prompt format clarity, rule-specific error messages, and maximum menu depth <= 3.
Test Type	Usability
Test Level	System
Preconditions	Standard running application.
Test Data	`All menu paths and error conditions.`
Execution Steps	1. Inspect prompt text for format guidance (e.g. YYYY-MM-DD, HH:MM). 2. Trigger validation errors and inspect message clarity. 3. Measure menu depth from Main Menu to leaf operations.
Expected Result	All prompts state expected format; error messages cite specific rule; maximum menu depth is <= 3 levels.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #102. CTest target: TC-NFR-05_UsabilityMenuDepth.

Test Case Specification: TC-NFR-06 — Architectural Class Isolation and File I/O Separation

Attribute	Specification Detail
Test Case ID	`TC-NFR-06`
Requirement ID	`HMS-NF-006`
Title	Architectural Class Isolation and File I/O Separation
Test Objective	Verify presence of 5 distinct OOP classes and confirm file I/O is isolated strictly to FileManager.
Test Type	Maintainability
Test Level	Static / Architectural
Preconditions	C++ source code in src/ and include/hms/.
Test Data	`Source code files.`
Execution Steps	1. Inspect class definitions: Patient, Doctor, Appointment, HospitalSystem, FileManager. 2. Scan codebase for fstream, ifstream, ofstream usage.
Expected Result	Five distinct classes verified; file-stream operations exist exclusively within FileManager.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #104. CTest target: TC-NFR-06_ArchitecturalClassIsolation.

Test Case Specification: TC-NFR-07 — Cross-Platform C++17 Compilation

Attribute	Specification Detail
Test Case ID	`TC-NFR-07`
Requirement ID	`HMS-NF-007`
Title	Cross-Platform C++17 Compilation
Test Objective	Verify clean compilation with -std=c++17 -Wall on Windows (MinGW/MSVC) and Linux (GCC).
Test Type	Portability
Test Level	Build
Preconditions	Repository cloned on target operating systems.
Test Data	`cmake -S . -B build && cmake --build build`
Execution Steps	1. Execute CMake configure and build on Windows. 2. Execute CMake configure and build on Linux. 3. Verify zero compiler errors.
Expected Result	Build succeeds with zero compilation errors and warnings on both platforms.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #107. Build-time CMake configuration verification; Linux GCC build validated via .github/workflows/linux-build.yml; Windows build validated with MinGW/MSVC (-std=c++17 -Wall). Lacks standalone CTest executable target.

Test Case Specification: TC-NFR-08 — Appointment Conflict Matrix Correctness

Attribute	Specification Detail
Test Case ID	`TC-NFR-08`
Requirement ID	`HMS-NF-008`
Title	Appointment Conflict Matrix Correctness
Test Objective	Verify 100% accuracy of appointment conflict detection across all 4 combination cases.
Test Type	Correctness
Test Level	Integration
Preconditions	Doctor D001 has appointment on 2026-11-15 at 10:00.

Test Data	`Case 1: Same doctor, same slot (Reject)`
	`Case 2: Same doctor, different time (Accept)`
	`Case 3: Different doctor, same slot (Accept)`
	`Case 4: Same doctor, same slot after cancellation (Accept)`

Execution Steps	1. Execute booking for Case 1. Verify rejection. 2. Execute booking for Case 2. Verify acceptance. 3. Execute booking for Case 3. Verify acceptance. 4. Cancel original appointment and execute Case 4. Verify acceptance.
Expected Result	All four conflict matrix scenarios evaluate with 100% accuracy.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #103. CTest target: TC-NFR-08_AppointmentConflictCorrectness.

Test Case Specification: TC-NFR-09 — Standalone Offline Operation Verification

Attribute	Specification Detail
Test Case ID	`TC-NFR-09`
Requirement ID	`HMS-NF-009`
Title	Standalone Offline Operation Verification
Test Objective	Verify application operates completely offline without network sockets or external database dependencies.
Test Type	Deployment
Test Level	System
Preconditions	Target execution host with network adapters disabled.
Test Data	`objdump -p hms.exe, execution of full functional smoke test offline.`
Execution Steps	1. Run objdump on hms.exe to verify no socket/network library links (ws2_32.dll, libcurl). 2. Run smoke tests on offline host.
Expected Result	Zero network imports verified; application runs with full capability in air-gapped environment.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #105, #116. CTest target: TC-NFR-09_StandaloneOffline (symbol verification via objdump); air-gapped execution validated on offline machine.

Test Case Specification: TC-NFR-10 — Human-Readable CSV File Interoperability

Attribute	Specification Detail
Test Case ID	`TC-NFR-10`
Requirement ID	`HMS-NF-010`
Title	Human-Readable CSV File Interoperability
Test Objective	Verify data files are valid delimiter-separated plain text with headers, readable in standard text editors.
Test Type	Interoperability
Test Level	System
Preconditions	patients.csv, doctors.csv, appointments.csv populated with records.
Test Data	`Direct text file inspection and schema comparison.`
Execution Steps	1. Open each CSV file in text editor. 2. Inspect header row against Section 3.3 specification. 3. Verify comma delimiters and newline termination.
Expected Result	All files conform to standard CSV specification with exact matching header columns.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #108. CTest target: TC-NFR-10_HumanReadableCSV.

14. Detailed Security Safeguards Test Cases (HMS-SR-001 to HMS-SR-008)

Test Case Specification: TC-SEC-01 — Accept-List Input Character Set Validation

Attribute	Specification Detail
Test Case ID	`TC-SEC-01`
Requirement ID	`HMS-SR-001`
Title	Accept-List Input Character Set Validation
Test Objective	Verify keyboard inputs are strictly validated against domain accept-lists before altering state.
Test Type	Security
Test Level	System
Preconditions	Application running at data entry prompts.
Test Data	`Inputs violating character domains: non-digit contacts, invalid names with symbols, malformed IDs`
Execution Steps	1. Attempt entering Patient ID with punctuation ('P-01!'). 2. Attempt entering Contact with alpha characters ('98765abcde'). 3. Observe rejection.
Expected Result	Inputs violating accept-list rules are rejected; no memory state or CSV file is altered.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #109. CTest target: TC-SEC-01_AcceptListInputValidation.

Test Case Specification: TC-SEC-02 — CSV Delimiter and Formula Injection Guard

Attribute	Specification Detail	
Test Case ID	`TC-SEC-02`	
Requirement ID	`HMS-SR-002`	
Title	CSV Delimiter and Formula Injection Guard	
Test Objective	Verify rejection of inputs containing commas, newlines, or formula execution prefixes (=, +, -, @).	
Test Type	Security	
Test Level	System	
Preconditions	Application running at Patient Name prompt.	
Test Data	`Input 1: 'Doe, John', Input 2: '=cmd	" /C calc"!A0', Input 3: '+12345', Input 4: '@SUM(A1:A5)'`
Execution Steps	1. Attempt registering patient with Name containing comma. 2. Attempt registering patient with Name starting with '='. 3. Attempt registering patient with Name starting with '+'.	
Expected Result	System rejects inputs displaying exact message 'Input contains illegal character (comma, newline, or formula prefix =, +, -, @)'.	
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)	
Execution Status	**Not Executed**	
Remarks / Evidence	PR #110. CTest target: TC-SEC-02_CSVFormulaInjectionGuard.	

Test Case Specification: TC-SEC-03 — Over-Length Input Buffer Safety

Attribute	Specification Detail
Test Case ID	`TC-SEC-03`
Requirement ID	`HMS-SR-003`
Title	Over-Length Input Buffer Safety
Test Objective	Verify system handles 10,000+ character console inputs safely without buffer overflows or heap corruption.
Test Type	Security / Robustness
Test Level	System
Preconditions	Application running at console input prompts.
Test Data	`Synthetic strings of 10,000 'A' characters.`
Execution Steps	1. Submit 10,000-character string to Patient Name prompt. 2. Submit 10,000-character string to Address prompt. 3. Observe system stability.
Expected Result	System safely consumes input, rejects value as exceeding maximum field bounds, and prompts again with zero memory corruption.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #111. CTest target: TC-SEC-03_OverLengthInputSafety.

Test Case Specification: TC-SEC-04 — Untrusted CSV File Validation

Attribute	Specification Detail
Test Case ID	`TC-SEC-04`
Requirement ID	`HMS-SR-004`
Title	Untrusted CSV File Validation
Test Objective	Verify data files are treated as untrusted input, validating every field upon load and skipping invalid rows.
Test Type	Security / Robustness
Test Level	Integration
Preconditions	Seeded CSV files containing injected invalid fields across doctor and appointment records.
Test Data	`Corrupted CSV datasets.`
Execution Steps	1. Place corrupted CSV files in data directory. 2. Launch application. 3. Verify only valid rows are instantiated in memory.
Expected Result	Malformed or invalid rows are rejected entirely without partial instantiation; application operates safely.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #112. Exercised via TC-PER-03_SkipMalformedCSV and TC-NFR-02_RobustStartup.

Test Case Specification: TC-PER-07 — Non-Destructive File Writing (Atomic Safety)

Attribute	Specification Detail
Test Case ID	`TC-PER-07`
Requirement ID	`HMS-SR-005`
Title	Non-Destructive File Writing (Atomic Safety)
Test Objective	Verify that previous file content is retained intact if a write operation fails.
Test Type	Security / Reliability
Test Level	Integration
Preconditions	Valid patients.csv exists; write failure simulated via read-only file permissions.
Test Data	`Attempt adding patient record.`
Execution Steps	1. Capture initial file byte content of patients.csv. 2. Set file read-only. 3. Trigger write operation. 4. Re-read and compare file content byte-for-byte.
Expected Result	Error reported; file content remains 100% byte-identical to original pre-operation state.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Official STP Table 7 test ID is TC-PER-07 (alias TC-SEC-05 in sprint notes). Atomic non-destructive write logic verified via CTest target TC-PER-07_HandleCSVWriteFailures (PR #113).

Test Case Specification: TC-APT-09 — Non-Deletion Audit Trail Integrity

Attribute	Specification Detail
Test Case ID	`TC-APT-09`
Requirement ID	`HMS-SR-006`
Title	Non-Deletion Audit Trail Integrity
Test Objective	Verify that cancelled appointments are preserved in appointments.csv for audit compliance.
Test Type	Security / Audit
Test Level	System
Preconditions	Appointment A0001 exists in appointments.csv.
Test Data	`Cancel A0001.`
Execution Steps	1. Verify row count of appointments.csv. 2. Cancel appointment A0001. 3. Inspect appointments.csv.
Expected Result	Row count remains identical; status column is updated from Booked to Cancelled; no physical deletion occurs.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Official STP Table 7 test ID is TC-APT-09 (alias TC-SEC-06 in sprint notes). Dedicated security CTest target TC-SEC-06_RetainCancelledAppointmentRecords added in PR #114.

Test Case Specification: TC-SEC-07 — Patient Data Privacy Minimization in Appointment Views

Attribute	Specification Detail
Test Case ID	`TC-SEC-07`
Requirement ID	`HMS-SR-007`
Title	Patient Data Privacy Minimization in Appointment Views
Test Objective	Verify appointment views display only essential IDs and Names, concealing contact, address, and medical notes.
Test Type	Security / Privacy
Test Level	System
Preconditions	Patient P001 (Contact: 9876543210, Address: 123 Main St, Note: HIV Positive) has appointment A0001.
Test Data	`View All Appointments (Option 1).`
Execution Steps	1. Select View All Appointments. 2. Inspect all displayed columns and values.
Expected Result	Display shows Appointment ID, Patient ID/Name, Doctor ID/Name, Date, Time, Status. Address, contact number, and medical note are omitted.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	PR #115. CTest target: TC-SEC-07_MinimizePatientDataInAppointmentViews.

Test Case Specification: TC-NFR-09 — Air-Gapped Network Isolation Verification

Attribute	Specification Detail
Test Case ID	`TC-NFR-09`
Requirement ID	`HMS-SR-008`
Title	Air-Gapped Network Isolation Verification
Test Objective	Verify zero network socket initialization or telemetry transmission during execution.
Test Type	Security / Isolation
Test Level	System
Preconditions	Compiled hms.exe binary.
Test Data	`Symbol table inspection via objdump.`
Execution Steps	1. Disassemble binary import directory. 2. Check for socket, connect, send, recv, WSAStartup symbols.
Expected Result	Zero network symbols found; application is certified for strict air-gapped operation.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Official STP Table 7 test ID is TC-NFR-09 (alias TC-SEC-08 in sprint notes). Verified via objdump network symbol inspection in test_standalone_offline.cmake (CTest target TC-NFR-09_StandaloneOffline) and PR #116.

15. Detailed Additional Sprint Test Cases (Supplementary Scope)

> [!NOTE]

> **Supplementary Project Scope ID Notice:** The test cases in this section validate capabilities and hardening introduced during Sprint 1 and Sprint 2 that extend beyond the frozen 54-requirement denominator of STP v1.0. They are formally tracked in Part B of the RTM.

Test Case Specification: TC-SEC-09 — Sanitize Filesystem Error Messages

Attribute	Specification Detail
Test Case ID	`TC-SEC-09`
Requirement ID	`HMS-SR-009`
Title	Sanitize Filesystem Error Messages
Test Objective	Verify error messages omit sensitive internal information such as local file paths, raw records, or exception text.
Test Type	Security / Privacy
Test Level	System
Preconditions	Application running under error trigger conditions.
Test Data	`Trigger missing file or write failure errors.`
Execution Steps	1. Cause a file write error. 2. Trigger CSV parsing error. 3. Inspect console error output.
Expected Result	Error message provides friendly explanation without exposing absolute paths (e.g. C:\Users\...), stack traces, or raw data.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Supplementary Project Scope ID — Not part of the original approved SRS/STP baseline. PR #117 (Issue #55). CTest target: TC-SEC-09_SanitizeErrorMessages.

Test Case Specification: TC-SEC-10 — Restrict File Operations to Designated Data Directory

Attribute	Specification Detail
Test Case ID	`TC-SEC-10`
Requirement ID	`HMS-SR-010`
Title	Restrict File Operations to Designated Data Directory
Test Objective	Verify application operates strictly within designated data folder and rejects directory traversal or external paths.
Test Type	Security / Path Traversal
Test Level	System
Preconditions	Application running in production mode.
Test Data	`Attempting path traversal strings (e.g. ../../secret.txt).`
Execution Steps	1. Inspect file path resolution in FileManager. 2. Attempt accessing files outside data directory.
Expected Result	System confines file I/O strictly to designated data directory; arbitrary paths rejected.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Supplementary Project Scope ID — Not part of the original approved SRS/STP baseline. PR #118 (Issue #56). CTest target: TC-SEC-10_RestrictDataDirectory.

Test Case Specification: TC-APT-11 — Appointment Cancellation Confirmation Workflow

Attribute	Specification Detail
Test Case ID	`TC-APT-11`
Requirement ID	`HMS-F-023-EXT`
Title	Appointment Cancellation Confirmation Workflow
Test Objective	Verify appointment summary display (Patient Name, Doctor Name, Date, Time, Status) and strict Y/N confirmation loop before cancelling.
Test Type	Positive / Negative Functional
Test Level	System
Preconditions	Active appointment A0001 exists with status Booked for Patient P001 and Doctor D001 on 2026-11-15 at 10:00.
Test Data	`Input sequence: A0001 -> 'maybe' -> 'N' -> A0001 -> 'Y'`
Execution Steps	1. Select Cancel Appointment. 2. Enter Appointment ID 'A0001'. 3. Inspect displayed summary block (Appointment ID, Patient Name, Doctor Name, Date, Time, Status). 4. Enter invalid confirmation 'maybe'. Observe error 'Invalid confirmation. Enter Y or N.' and re-prompt. 5. Enter 'N'. Verify output 'Cancellation cancelled by user.' and appointments.csv remains unmodified. 6. Re-initiate cancellation for 'A0001' and enter 'Y'.
Expected Result	Summary displays all key appointment fields. Entering 'N' aborts without modifying records. Confirming 'Y' cancels appointment with message 'Appointment cancelled successfully' and persists Cancelled status to appointments.csv.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Supplementary Project Scope ID — Not part of the original approved SRS/STP baseline. PR #122 (Issue #120). Tested in tests/test_cancel_appointment.cpp.

Test Case Specification: TC-APT-12 — Filter Unknown Entity in Appointment Views

Attribute	Specification Detail
Test Case ID	`TC-APT-12`
Requirement ID	`HMS-F-022-EXT`
Title	Filter Unknown Entity in Appointment Views
Test Objective	Verify patient and doctor existence is validated before querying appointments when filtering by ID.
Test Type	Positive / Negative Functional
Test Level	System
Preconditions	Patient P001 has appointments; P002 has no appointments; P999 is unregistered. Doctor D001 has appointments; D002 has no appointments; D999 is unregistered.
Test Data	`Filter by Patient: P001, P002, P999; Filter by Doctor: D001, D002, D999`
Execution Steps	1. Filter appointments by Patient ID 'P999'. Observe output. 2. Filter appointments by Patient ID 'P002'. Observe output. 3. Filter appointments by Patient ID 'P001'. Observe output. 4. Repeat for Doctor IDs D999, D002, D001.
Expected Result	Nonexistent IDs (P999, D999) display exact error 'Patient not found' or 'Doctor not found'. Registered entities with zero appointments (P002, D002) display 'No appointments found.'. Matching appointments displayed chronologically.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Supplementary Project Scope ID — Not part of the original approved SRS/STP baseline. PR #123 (Issue #121). CTest target: TC-APT-12_FilterUnknownEntity (69 assertions passed).

Test Case Specification: TC-ID-01 — Duplicate Patient and Doctor Identity Validation

Attribute	Specification Detail
Test Case ID	`TC-ID-01`
Requirement ID	`HMS-F-001/008-EXT`
Title	Duplicate Patient and Doctor Identity Validation
Test Objective	Verify heuristic detection of duplicate human individuals (Patient: normalized name + contact; Doctor: normalized name + contact + specialization) and explicit Y/N confirmation loop.
Test Type	Data Integrity
Test Level	System / Integration
Preconditions	Patient P001 ('John Doe', contact '9876543210') exists in patients.csv. Doctor D001 ('Dr. Alice Smith', spec 'Cardiology', contact '9876543211') exists in doctors.csv.
Test Data	`Patient flow: P002, 'john doe', '9876543210', confirm 'N' then 'Y'. Doctor flow: D002, 'alice smith', 'Cardiology', contact '9876543211', confirm 'N' then 'Y'.`
Execution Steps	1. In Patient Registration, enter ID P002, name ' john doe ', age 30, gender M, contact '9876543210', address '456 Oak St', note 'None'. 2. Observe warning 'Warning: Potential duplicate patient record: matches existing patient with ID P001' and prompt 'Proceed with registration? (Y/N): '. 3. Enter 'N'. Verify output 'Registration cancelled.' and patients.csv remains byte-identical. 4. Re-enter same data and enter 'Y'. Verify registration succeeds and saves P002. 5. Register patient with name 'John Doe' but different contact '9111111111'. Verify no warning displayed. 6. In Doctor Registration, enter ID D002, name 'alice smith', spec 'Cardiology', dept 'Cardio', contact '9876543211', avail 'Mon-Fri 09:00-17:00'. 7. Observe warning 'Warning: Potential duplicate doctor record: matches existing doctor with ID D001' and prompt. 8. Enter 'N'. Verify cancellation. Enter 'Y' on re-attempt. Verify save. 9. Add doctor with name 'Dr. Alice Smith', contact '9876543211' but different specialization 'Neurology'. Verify no warning displayed.
Expected Result	System detects identity similarity using normalized name + contact (patient) and normalized name + contact + specialization (doctor). Displays exact warning with existing ID; entering 'N' cancels registration leaving CSV files byte-identical; entering 'Y' completes registration.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Supplementary Project Scope ID — Not part of the original approved SRS/STP baseline. PR #125 (Issue #124). CTest target: TC-ID-01_DuplicateIdentityValidation.

Test Case Specification: TC-VAL-01 — Universal Immediate Field-Level Validation and Interactive Retry

Attribute	Specification Detail
Test Case ID	`TC-VAL-01`
Requirement ID	`HMS-F-003/010/018-EXT`
Title	Universal Immediate Field-Level Validation and Interactive Retry
Test Objective	Verify that invalid inputs prompt immediate retry of the same field without restarting the entire form.
Test Type	Usability / Robustness
Test Level	System
Preconditions	Application running at interactive console data entry forms.
Test Data	`Invalid entries followed by valid corrections across patient, doctor, and appointment forms.`
Execution Steps	1. In Patient Registration, enter invalid Age '150'. Observe rule-specific error message. 2. Verify system prompts for Age again immediately without losing previously entered ID and Name. 3. Enter valid Age '30'. Verify system advances to Gender. 4. Enter invalid Gender 'Z'. Observe error and immediate retry. 5. In Appointment Booking, enter invalid Date '2026-02-30'. Observe immediate error and re-prompt for Date.
Expected Result	System validates each field immediately using the readValidatedField template, displays rule violation, and prompts for immediate retry while preserving previously validated fields.
Actual Result	Not Executed (Document update baseline; historical test run verified in Sprint 1/2)
Execution Status	**Not Executed**
Remarks / Evidence	Supplementary Project Scope ID — Not part of the original approved SRS/STP baseline. PR #127 (Issue #119). Note: No dedicated standalone CTest target executable; verified across interactive test targets (TC-APT-01_BookAppointment, TC-APT-09_CancelAppointment, TC-DOC-01_AddDoctor, TC-PAT-05_PatientDetailsAndRegistrationPersistence, TC-PAT-06_UpdatePatientDetails, TC-DOC-06_UpdateDoctorDetails).

16. Sprint 1 and Sprint 2 Change Traceability

Across Sprint 1 (19–23 October 2026: 31 merged PRs #60–#90), Sprint 2 (26–30 October 2026: 31 merged PRs #91–#118, #122, #123, #125), and immediate post-sprint follow-up (30/31 October 2026: PR #127), a total of 63 pull requests were merged into `main`. The table below outlines how these sprint implementations evolved the system's test coverage:

Functional Area	Baseline STP Coverage	Sprint 1 & Sprint 2 Enhancements	Resulting Test Case Specifications
Patient Management	Basic registration, duplicate check, field checks, search, view	Case-insensitive ID checks, whitespace trimming, interactive field updates, immutable ID, duplicate identity detection, immediate field retry	`TC-PAT-01` to `TC-PAT-07`, `TC-ID-01`, `TC-VAL-01`
Doctor Management	Basic addition, duplicate check, search, view	Case-insensitive ID checks, interactive field updates, immutable ID, duplicate doctor identity heuristics, immediate field retry	`TC-DOC-01` to `TC-DOC-07`, `TC-ID-01`, `TC-VAL-01`
Appointment Management	Basic booking, FK rejection, conflict check, view, cancel	Leap year calendar checks, double-booking rejection, slot rebooking after cancellation, chronological sorting, confirmation workflow, entity validation on filter	`TC-APT-01` to `TC-APT-10`, `TC-APT-11`, `TC-APT-12`
Persistence & File I/O	Startup load, missing files, malformed rows	Duplicate & orphan row skipping, immediate flush, atomic non-destructive write safety, restart data retention	`TC-PER-01` to `TC-PER-07`
Console UI	Menu hierarchy, invalid option rejection	Process return code 0, standardized outcome messaging, transactional state protection, menu depth ≤ 3	`TC-UI-01` to `TC-UI-05`
Non-Functional Quality	Latency, robustness, class isolation, offline	1,000-record performance benchmarks, air-gapped objdump verification, 20+ input crash resilience vectors	`TC-NFR-01` to `TC-NFR-10`
Security & Privacy	Accept-list validation, injection guards, privacy	Filesystem error message sanitization, strict data directory access, over-length buffer safety, audit retention	`TC-SEC-01` to `TC-SEC-10`

---

17. Updated Requirements Traceability Matrix Summary

The complete, bidirectional Requirements Traceability Matrix is provided in the companion document:

[`02_Updated_Requirements_Traceability_Matrix.md`](02_Updated_Requirements_Traceability_Matrix.md).

- **Part A (Baseline Scope):** Maps all 54 baseline requirements (`HMS-F-001` through `HMS-SR-008`) to official STP test cases, updated test cases, CTest targets, and source files.

- **Part B (Supplementary Scope):** Maps all 6 sprint additions (`HMS-SR-009`, `HMS-SR-010`, `HMS-F-022-EXT`, `HMS-F-001/008-EXT`, `HMS-F-023-EXT`, `HMS-F-003/010/018-EXT`) to dedicated test cases and CTest targets.

---

18. Test Case Design Coverage Summary

Scope Category	Total Requirements	Test Cases Specified	Design Coverage %
Baseline Functional (HMS-F)	36	36	**100.0%**
Baseline Non-Functional (HMS-NF)	10	10	**100.0%**
Baseline Security (HMS-SR)	8	8	**100.0%**
Supplementary Sprint Scope	6	6	**100.0%**
Total Project Scope	**60**	**60**	**100.0%**

---

19. Known Testing Gaps and Implementation Notes

1. **Build-Time vs Runtime Verification for `HMS-NF-007`:** Cross-platform portability is enforced at compile time via CMake compiler flags (`-std=c++17 -Wall`) and verified across Windows and Linux CI builds. It does not have an independent `.exe` CTest runtime executable target.

2. **CTest Naming Alignments:**

- In STP Table 7, duplicate/orphan record handling is numbered `TC-PER-04`. In CTest, the target is named `TC-PER-08_SkipDuplicateAndOrphanRecords`.

- In STP Table 7, data retention after restart is numbered `TC-PER-06`. In CTest, the target is named `TC-NFR-03_DataRetention`.

- In STP Table 7, immediate persistence is numbered `TC-PER-05`. In CTest, it is tested under `TC-PAT-05_PatientDetailsAndRegistrationPersistence`.

- In STP Table 7, clean exit return code is covered under `TC-UI-05`. In CTest, two separate targets verify this: `TC-UI-05_ExitCleanly` and `TC-UI-05_ExitProcessCode`.

3. **Execution Status Baseline:** For this Deliverable Part-3 documentation activity, all test specifications are marked **Not Executed** to establish a clean, honest baseline for the upcoming execution campaign, while recording historical pass status from Sprint 1 and Sprint 2 merges.

---

20. Approvals and Sign-Off

Role	Name	Signature / Status	Date
QA Lead / Student	Hrushikesh Karnam	Submitted by Project Team	02-Nov-2026
Test Engineer / Student	Muruli H	Submitted by Project Team	02-Nov-2026
Developer Lead / Student	Mandagandla Jhansi	Submitted by Project Team	02-Nov-2026
Support Lead / Student	Neeta R	Submitted by Project Team	02-Nov-2026
Course Coordinator / Evaluator	Mini-Project Evaluator	Pending Course Evaluator Review	—